



Explaining software fault predictions to spreadsheet users[☆]

Adil Mukhtar^{a,*}, Birgit Hofer^a, Dietmar Jannach^b, Franz Wotawa^a

^a Graz University of Technology, Austria

^b University of Klagenfurt, Austria

ARTICLE INFO

Article history:

Received 2 December 2022

Received in revised form 7 March 2023

Accepted 9 March 2023

Available online 16 March 2023

Keywords:

Fault prediction

Explanations

Spreadsheets

User study

Explainable AI

ABSTRACT

A variety of automated software fault prediction techniques was proposed in recent years, in particular for the important class of spreadsheet programs. Software fault prediction techniques commonly create ranked lists of “suspicious” program statements for developers to inspect. Existing research, however, suggests that solely providing such ranked lists may not always be effective. In particular, it was found that developers often seek for *explanations* for the outcomes provided by a debugging tool and that such explanations may be key for developers to trust and rely on the tool. Research on *how to explain* the outcomes of fault prediction techniques, which are often based on complex machine learning models, is scarce, and little is known regarding how such explanations are perceived by developers. With this work, we aim to narrow this research gap and study the perception of different forms of explanations by spreadsheet users in the context of a machine learning based fault prediction tool. A between-subjects user study (N=120) revealed significant differences between the explored explanation styles. In particular, we found that well-designed natural language explanations can indeed help users better understand why certain spreadsheet cells were marked by the debugging tool and that such explanations can be effective to increase the users' trust compared to a black box system.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Over the last decades, a multitude of automated techniques were proposed that aim for supporting software developers prevent, localizing, or even correcting faults in their programs. Software fault prediction is a particular class of such approaches, which aim to pinpoint fault-prone parts of a program as early as possible, ideally even before system testing. Technically, the prediction task is often framed as a supervised machine learning problem, where software-related metrics in combination with past fault data are used to estimate the likelihood that a given part of a software artifact is faulty, see [Catal \(2011\)](#) for a survey. In recent years, such learning-based fault prediction methods – as well as various other advanced debugging techniques – were also successfully applied to the important class of spreadsheet programs ([Koch et al., 2019](#)). Nowadays, spreadsheet programs are used widely in organizations for various forms of data analysis

and decision-making, and it is not uncommon that business-critical decisions are based on calculations in spreadsheets. Thus, appropriate quality assurance techniques for spreadsheet programs – which are the focus of our work – are of high practical importance.¹

Today's research on automated fault prediction and fault localization – both for spreadsheets and traditional programs – is largely based on data-based (“offline”) experiments, where machine learning models are trained and assessed without involving humans in the evaluation process. The central task in such machine learning approaches is to predict the likelihood of a certain part of a given software, e.g., a statement in a program or a formula in a spreadsheet, being faulty. Ultimately, the main outcome of such techniques is typically an ordered list of program elements that are considered suspicious. One fault prediction technique is then considered better than another one if it (a) more often ranks the true faults higher in this list and (b) includes fewer “false alarms” at the top places. Both aspects can be evaluated with common accuracy and ranking metrics from the field of information retrieval, e.g., with precision, recall and Mean Reciprocal Rank ([Miryeganeh et al., 2021](#)).

¹ See <http://www.eusprig.org/horror-stories.htm> for a list of “horror stories”, where faults in spreadsheets led to significant financial losses for organizations.

[☆] Editor: Christoph Treude.

* Correspondence to: Inst. f. Software Technology, Graz University of Technology, Inffeldgasse 16b/EG, 8010 Graz, Austria.

E-mail addresses: amukhtar@ist.tugraz.at (A. Mukhtar), bhofer@ist.tugraz.at (B. Hofer), dietmar.jannach@aau.at (D. Jannach), wotawa@ist.tugraz.at (F. Wotawa).

While such an evaluation approach is widely used in the academic literature, Parnin and Orso (Parnin and Orso, 2011) questioned if the underlying assumptions of this approach hold, and if we can be sure that automated debugging tools are helping programmers in practice. One main outcome of their studies was that some central assumptions may not always hold in practice. Programmers for example may not necessarily inspect a given list of potentially faulty statements strictly in order; also, they might not identify a statement as being faulty, even though it is presented as potentially faulty by the debugging tool. Different remedies to these issues were proposed, including in particular the use of alternative evaluation metrics, support for program comprehension, the provision of ecosystems that support the entire toolchain, and more user studies (Ang et al., 2017).

In the context of program comprehension, one important direction to improve the practical usefulness of automated debugging techniques lies in the provision of *explanations*. In a recent study on defect prediction methods, Wan and colleagues (Wan et al., 2020) found that several respondents of their survey explicitly mentioned it to be a desirable feature if the tool could explain “[...] why an error is considered an error” or “[t]ell me why certain lines are potentially wrong”. In this context, a largely open question is *how* a system should explain such aspects to its users: What kind of information should be displayed? How much detail should be presented? Should the information be conveyed through natural language or some form of visualization?

In this work, we investigate these questions for the particular case of spreadsheet debugging. Specifically, we investigate different forms of explanations for the machine-learning-based spreadsheet debugging approach proposed in Koch et al. (2019). Given a spreadsheet as input, this debugging technique returns a “suspiciousness” score for each formula in the spreadsheet.² The data-based experiments in Koch et al. (2019) indicate that the method is highly effective to identify faults in spreadsheets. Continuing this research, our present goal is to explore in which ways explanations can help to further increase the usefulness of the automated fault prediction method. In particular, we aim to assess to what extent different forms of explanations help developers to understand why a given formula is faulty. Moreover, we are interested in the effects of providing explanations to spreadsheet users³ in terms of the quality perception of the debugging tool, e.g., perceived usefulness, usability, transparency, and trust. Therefore, the main research questions in this work are as follows.

- **RQ-1:** *Can explanations for individual fault predictions help spreadsheet users obtain a better understanding of the spreadsheet program and its faults?*
- **RQ-2:** *Can explanations help to improve the quality perception of the debugging tool by spreadsheet users and the trust in the system?*

To answer these research questions, we conducted a between-subjects user study (N=120), where participants were presented with faulty spreadsheets in which several formulas were highlighted as being suspicious. We then provided different forms of explanations to the participants and asked them to (a) describe why they think that a given formula might be faulty and (b) report

² We note here that we focus on *faulty formulas* in this work. Extending the scope to erroneous input data or constants is possible in principle. For more general discussions of faults (errors) in spreadsheets, we refer to Panko (1998, 2008).

³ In the context of our work, we use the term “spreadsheet users” to refer to people who use spreadsheet on a regular basis, who understand the basic concept of entering formulas in cells, and who are able to create or modify spreadsheets by themselves.

on their perception of the debugging tool in different dimensions. The results of our study indicate that providing explanations can indeed be beneficial compared to a black box system both in terms of program comprehension and other quality dimensions. We also observed that the particular form of explaining matters.

Overall, with our work, we *contribute to a better understanding of the value of explanations in the context of automated fault prediction techniques for the class of spreadsheet programs*. To our knowledge, no prior work exists that examines such questions in the domain of spreadsheets. To what extent our findings generalize to other types of programs, however, still has to be explored. While we could merely see spreadsheets as a particular type of software, one crucial difference to general software development is that the users and creators of spreadsheets often have no formal education in software engineering. Moreover, there are typically no *systematic* testing and debugging procedures implemented during spreadsheet development, and automated fault localization and debugging support in today’s spreadsheet environments is still limited. These aspects make the design of the technical solutions to present explanations to spreadsheet users particularly challenging, because appropriate and concise ways of conveying the relevant information, e.g., through text summarizations, visualizations or explanation-by-example (Barredo Arrieta et al., 2020), must be designed to ensure the understandability of the explanations for this particular type of users.

On a more general level, our work may also be seen as a contribution to the area of Explainable AI (XAI), where one main goal is to make the underlying reasoning of an AI-based system transparent to developers and end users. In the context of automated fault prediction, explainability turns out to be particularly challenging when the underlying machine learning models become increasingly complex with the use of modern deep learning architectures, see Omri and Sinz (2020) for an overview.

The paper is organized as follows. We discuss related work next in Section 2. Our research methodology and the details of our experimental design are described in Section 3. In Section 4 we discuss the obtained results and practical implications in detail. The paper ends with a summary and an outlook on future works in Section 6.

2. Related work

The related works discussed in this section cover software fault prediction techniques and the use of explanations in the context of software engineering and in particular debugging. In the following, we outline recent work on fault prediction, discuss existing studies with humans in this domain, and elaborate on research dealing with explanations given to users with the aim of improving usability and trust.

2.1. Software fault prediction methods

In software development and maintenance, we spend a lot of effort that is related to the detection of failures and the identification of their corresponding root causes. Hence, there has always been an interest in automating debugging, which aims at identifying those parts of a program that potentially induce failures as early as possible. For a recent survey on software fault localization, we refer to Wong et al. (2016). In contrast to regular debugging activities in practice, software fault prediction, also called software defect prediction, does not require passing and failing test executions. Instead, software fault prediction utilizes information regarding programming patterns and metrics for estimating whether or not certain parts of a program are faulty. In the following, we summarize related research focusing on software fault prediction only.

We can view software fault prediction as an extension to the *checking approach* to debugging. According to (Ducasse, 1993), a checking strategy “*parses programs and searches for language-dependent errors*,” where we consider program parts suspicious when they are not conforming to well-formedness rules or conform to error rules. In contrast to checking, software fault prediction goes beyond simple classification and assigns suspiciousness values to potentially faulty program parts. These suspiciousness values should correspond to the likelihood of faultiness and allow ranking of program parts.

For finding a function that maps programming patterns or metrics to a suspiciousness value, most recent work on software fault prediction relies on machine learning, e.g., see Shepperd et al. (2014), where we use information regarding patterns or metrics and their effect on the behavior to learn the prediction function. A variety of different machine learning approaches has been used in the past, e.g., neural networks (Manjula and Florence, 2019), Bayesian networks (Okutan and Yildiz, 2014), and association rule mining (Czibula et al., 2014) among others. There are also several surveys which deal with general fault prediction (Catal, 2011; Radjenović et al., 2013; Wahono, 2015; Li et al., 2018; Rathore and Kumar, 2019; Thota et al., 2020), and the application of machine learning to fault prediction (Malhotra, 2015; Omri and Sinz, 2020; Li et al., 2020; Matloob et al., 2021).

In the context of spreadsheets, Koch and colleagues (Koch et al., 2019) successfully applied machine learning for software fault prediction based on metrics. In their paper, the authors compared different machine learning approaches for predicting the faultiness of spreadsheet cells based on several metrics. For a survey on known spreadsheet metrics, we refer the reader to Hofer et al. (2021).

Beyond the spreadsheet domain, there are several studies which evaluate and compare different fault prediction methods. The most recent studies include the following. Razu et al. Ahmed et al. (2020) compared several machine learning techniques for fault prediction using well-known evaluation benchmarks. He and colleagues (He et al., 2015) describe and evaluate an approach for computing a reduced set of metrics for estimating defects in software. Choudhary et al. (2018) introduced change metrics for software fault prediction. The commonality of these studies is the use of program collections for determining the prediction accuracy of the underlying software fault prediction approaches. They do not consider user-related characteristics, which include the perceived usefulness of predictions.

To the best of our knowledge, there is only little research that considers human-centric evaluation approaches for automated debugging tools, including (Ko and Myers, 2004, 2008; Burg et al., 2013; Parnin and Orso, 2011; Xie et al., 2016; Xia et al., 2016). Most of these papers report results obtained from user studies which either involve students or a limited number of developers. For spreadsheet debugging, the situation is similar. Papers dealing with user studies like (Aurigemma and Panko, 2010; Ruthruff et al., 2006; Jannach et al., 2019; Mukhtar et al., 2022) specifically concentrate on user-related characteristics of one particular tool. The content of this paper is distinct from previous work, elaborating on the effects of different ways a spreadsheet debugging tool passes explanations to the user.

2.2. Explanations

In an informal way, an explanation can be seen as a narrative answering the question of why particular facts (e.g., an event or a decision) are happening. Such a narrative may explain things in terms of causality, i.e., tracing back the fact to root causes from which we can deduce the particular fact. Although it is usually the case that one person (i.e., the explainer) is explaining

a decision or event to another person (i.e., the explainee), adding similar capabilities to engineered systems has been of growing interest recently. This increase in interest might be due to the fact of increasing automation, which relies more and more on AI technology. To gain trust in decisions taken by AI, users are often interested in their origins, which leads to the field of explainable AI (XAI). Miller (2019) provides an excellent discussion on explainability and the need for considering research in philosophy, psychology, and cognitive science when constructing XAI. In his paper, Miller distinguishes two complementary approaches for AI-based systems: “(1) *generating decisions in which one of the criteria taken into account during the computation is how well a human could understand the decisions in the given context, which is often called interpretability or explainability*; and (2) *explicitly explaining decisions to people, which we will call explanation*.” In this paper, the context is more related to the first approach, for which we discuss related research in more detail.

In the context of software debugging, a widely cited study from Parnin and Orso (2011) questioned the applicability of debugging tool in real life. One of the main implications from their work is to improve the explanatory capabilities of debugging tools. They found that expert programmers are faster with tool support, and that such debugging tools do not provide enough explanations when performing harder tasks. Many, if not most, of the fault localization and software fault prediction tools described in research articles rank the faulty components of a program in a list based on the suspiciousness score. These faulty components can be explained depending on the underlying methodology, however, in machine learning approaches the explanations are often generated through heuristic methods.

In a shallow way, we may also consider a given ranked list as an explanation for a bug or faulty proneness of statements. Such a list provides information on which part of a program is more likely to lead to misbehavior or a potential threat. Of course, it does not explain the ranking itself. We can extract such an explanation from the underlying ranking method. We only need to state how to compute all the ranking scores. However, a good explanation for any element in the ranked list would indicate why it is considered faulty and how we can distinguish it from the others. Parnin and Orso mentioned that the ranked list is not leading to a sufficient bug understanding and require improved explanatory capabilities. In this paper, we contribute to this challenge comparing different kinds of explanations given to users for bug candidates.

It is worth mentioning that Parnin and Orso (Parnin and Orso, 2011) also provided a survey on user studies to evaluate automated debugging tools. In this study, the authors discuss the work (Ko and Myers, 2004, 2008) by Ko and Myers on introducing the *Whyline* tool that allows to answer *why* and *why not* questions. This kind of explanation seems to improve the understanding of programs, and to increase debugging efficiency substantially. The *Whyline* tool goes beyond ranked lists as explanations.

2.3. Beyond ranked lists

Besides the mentioned *Whyline* tool, there is other related work that deals with explanations beyond ranked lists. This include early work on tutoring systems (Murray, 1988) for novice programmers. Recent works include (Fey et al., 2010) and (Groce et al., 2006). Fey and colleagues (Fey et al., 2010) combine fault localization with explanations that are based on counterexamples and their traces. Counterexamples help programmers to understand how and why a program breaks. Fey and colleagues also presented the results of an experimental study showing that the combination of localization and explanation improves debugging.

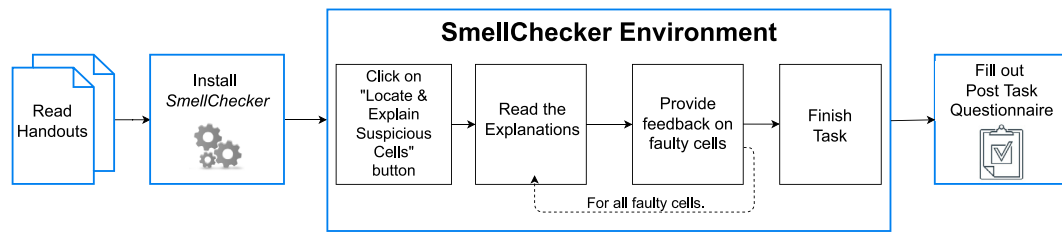


Fig. 1. Overall flow of the experiment.

However, the authors provide no user study. Groce et al. (2006) present a debugging approach that utilizes counterfactuals and causality for explanations. The approach is also based on counterexamples and distance metrics between passing and failing runs. In our work, we contribute to improving explanations beyond providing ranked lists. In particular, our aim is to find out which information is helpful for users to improve their understanding of the origin of faults.

3. Methodology – experiment design

In this section, we summarize our research methodology and provide details of our experimental design. Let us recall our research goals here, which are to investigate (i) to what extent different types of explanations help users to understand faults in a given spreadsheet program (RQ-1), and (ii) how different types of explanations affect the quality perception of the debugging tool (RQ-2).

3.1. Experiment overview

To address the research questions, we designed a user study in which participants interacted with an online version of Microsoft's Excel spreadsheet tool, which we extended with a plug-in component for debugging called SMELLCHECKER (Koch et al., 2019). The main functionality of the debugging plug-in is to automatically highlight formula cells in a spreadsheet that are considered "suspicious" based on a machine learning approach. For the purpose of the study, we enriched the SMELLCHECKER tool with the capability to provide different forms of explanations to users. These explanations indicate, in different ways, why the tool considered a given formula to be potentially faulty.

All participants of our study were directed to an online spreadsheet containing three faults, and these faults were correctly highlighted⁴ by the SMELLCHECKER tool. The participants in each treatment group in our between-subjects user study were additionally provided with one form of explanation. The control group was only shown the "suspiciousness score" computed by the tool without further explanation. The task of the participants was then to inspect the cells that were highlighted (and explained) by the tool and to describe in their own words why the formula may be faulty. An analysis of these responses through human judges will help us answer RQ-1 on the fault understanding of participants when given different types of explanations.

After the participants had completed this main task, they were forwarded to a comprehensive questionnaire. In this questionnaire, the participants informed about various facets of their

subjective perceptions regarding the tool, e.g., in terms of usefulness and usability, and the debugging process. This questionnaire allows us address RQ-2 on subjective user perceptions when provided different forms of explanations.

3.2. Detailed experiment flow

The more detailed flow of the experiment is illustrated in Fig. 1. In a preparatory step, participants read the instructions that were provided online to them and which explained their tasks and how to install the plug-in in the online version of Microsoft Excel. The descriptions of the task and the user manual for the tool were tailored for the different treatment groups, as they saw different types of explanations. We provide all materials used in the study online for reproducibility.⁵

After having provided informed consent and after the participants had installed the SMELLCHECKER tool, they were performing the main task in the spreadsheet environment. A screen capture of how the environment looked like in the control condition – the one in which only the suspiciousness score is displayed without further explanations – is shown in Fig. 2. Here, we can see that cell I34 was highlighted by the tool, and a (high) suspiciousness score of 80.92% is reported for this cell. Note that the annotations for the faulty cells shown in Fig. 2 were not part of the user interface, but introduced here to better illustrate the problems in the spreadsheet.

The main task for the participants, as mentioned, was to study the information provided by the tool for each cell and then fill a free-text input box for each highlighted cell, describing what they think might be the issue with the formula in the cell. Once this task was done for all highlighted cells the participants were forwarded to the post-task questionnaire. Note that all participants saw the same spreadsheet with the same highlighted cells.

3.3. Studied explanation types

The design space for providing explanations for the outputs of a machine learning based system is generally rich. Typically, there is a multitude of possible ways to explain things to users not only in terms of what *kind of content* is shown, but also in terms of *how* it is presented, e.g., in the form of text or as a visualization.⁶

Overview of explanation types. In our study, we investigated the effects of three different ways of explaining to the study participants why a cell is highlighted, plus a blackbox approach. Since we adopt a between-subjects design, these forms of explanations correspond to the treatment and the control group.

⁴ In our experiment, the fault prediction tool therefore did not return "false positives", i.e., it did not mark cells as suspicious, which did not actually contain a fault. We note that dealing with false positives is a challenge that concerns the fault prediction task that precedes the explanation phase. Improving the fault prediction accuracy in our case could for example be achieved by incorporating additional types of smells in the prediction model, including those discussed in Dou et al. (2014), Cheung et al. (2016), Hermans et al. (2012).

⁵ <https://doi.org/10.5281/zenodo.7703947>.

⁶ See for example (Nunes and Jannach, 2017) for a taxonomy of possible ways of explaining the outputs of a machine learning system in the context of decision support systems and recommender systems.

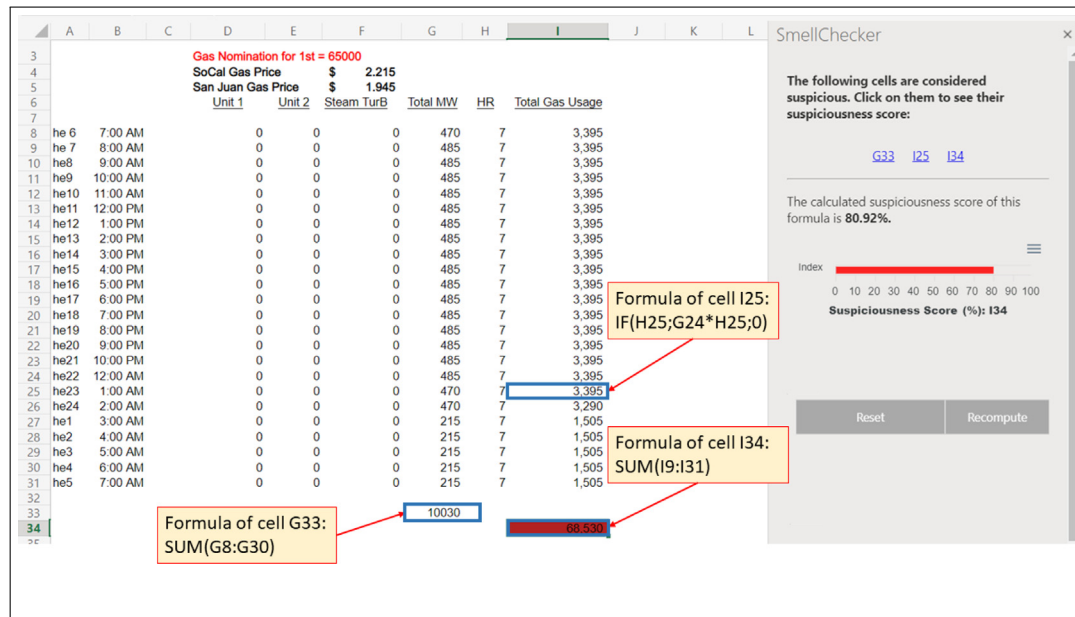


Fig. 2. SMELLCHECKER marks cells as faulty based on the suspiciousness score (Mukhtar et al., 2022).

1. **BLACKBOX:** In this condition, the debugging tool acts as a black box and only displays a suspiciousness score. This is the implementation of SMELLCHECKER as reported in Koch et al. (2019), and no information is provided by the tool in terms of how this score is computed or how it should be interpreted, cf. Figs. 2 and 3(a).
2. **VISUALIZATION:** This form of explanation is based on the visualization of feature importance values. The machine learning approach underlying SMELLCHECKER is based on a set of 64 metrics that are derived from the structure of a given spreadsheet and which serve as predictor variables for the fault probability of a given target cell, see Koch et al. (2019). To determine the feature importance values, we rely on the widely used LIME (Local Interpretable Model-agnostic Explanations) (Ribeiro et al., 2016) approach; see below for additional details. Fig. 3(b) shows how the feature importance values of the most relevant features (as per LIME) are visualized in a diagram.
3. **NATURALLANGUAGE:** These explanations are also based on feature importance values determined with LIME. This time, however, we do not display them visually, but show natural language explanations of each feature (predictor variable). To that purpose, we developed corresponding spreadsheet-independent natural language explanations for each predictor variable. An example of such an explanation is shown in Fig. 3(c).
4. **HUMANEXPLANATION:** Finally, we included an explanation that corresponds to how a knowledgeable human might describe the problem(s) for a given formula. The explanations shown in this condition thus serve as a form of upper limit of the quality of the explanations that could be provided by a tool. We iterate that these explanations were manually developed by us for the particular spreadsheet used in the experiment and that these explanations are not based on or restricted to the top features identified through LIME. An example of such an explanation is provided in Fig. 3(d).

Design rationale and background. Overall, our goal was to cover a diverse set of approaches to explanation, including both textual and visual ones. We consider the BLACKBOX method as a lower

bound (“no explanations”), and the HUMANEXPLANATION version represents a knowledgeable and precise explanation as provided by a human expert. The lower and upper bounds, therefore, let us assess where the automated explanations are located on this range. Note that the provision of “suspiciousness score”, even when this score is not explained to users in detail, might be considered as a form of explanation as well. However, in the context of our research goals, the score does not carry any information that would help participants understand *why* a certain formula might be faulty.

LIME is a recent and very popular technique to explain the prediction of any classifier in terms of its features. The outcome of LIME is a linear model with coefficient weights assigned to each feature, i.e., product metric in our case. Technically, these weights are obtained by slightly tweaking the feature values for a given learning instance in a way that they are still close to the original values, but the prediction of the base model changes. Ultimately, those features that have little impact on the outcome of the base model after tweaking their values are considered less important. An alternative to LIME would be to use SHAP (Lundberg and Lee, 2017) values. For our work, we, however, found that LIME explanations can be more efficiently computed. Moreover, LIME explanations are particularly helpful to explain one single prediction of a model. Finally, both for the VISUALIZATION and NATURALLANGUAGE explanations, we relied on this approach for explaining the predictions of SMELLCHECKER.

To keep the complexity of the VISUALIZATION and NATURALLANGUAGE explanations simple, we only considered the three most important features for a given formula according to the LIME model. In the VISUALIZATION approach, we presented users with a descriptive name of the feature and a bar chart that illustrated the value of the corresponding coefficient weight. For the NATURALLANGUAGE explanation, we displayed natural language descriptions of the top three features to the users. These descriptions were manually curated by us for the purpose of the study. Note that these descriptions characterize the respective product metric *in general*, and do not rely on specifics of the given spreadsheet, i.e., they can be used for any spreadsheet. The final descriptions used in our study are based on the descriptions of the 64 product metrics proposed earlier in Koch et al. (2019). The HUMANEXPLANATION descriptions, in contrast, are specific to the

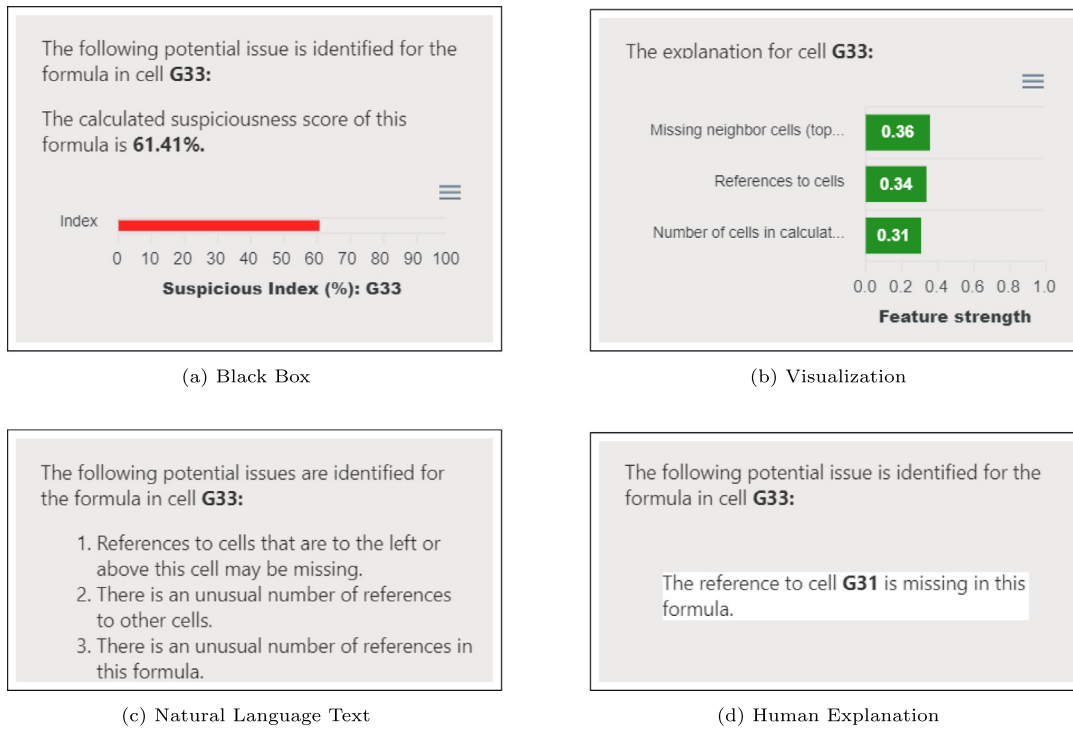


Fig. 3. Provided explanations.

given spreadsheet and *exactly* explain the fault in a given formula, e.g., “The reference to cell **G31** is missing in this formula.”. The exact wording of all relevant explanations can be found in Section 3.5, where we provide all relevant study material.

An important aspect to note here is that while the HUMANEXPLANATION explanations were *exact and accurate* (by design), this is not necessarily the case for the automated explanations based on LIME. In particular, none of our study’s top three features always point to the actual problem. In other words, the study participants were sometimes confronted with explanations, which could also contain elements that were actually not required or pointing in the right direction. This is important to mention because, in real-world applications, there is no guarantee that LIME explanations are always exact.

3.4. Measurement method

In this subsection, we discuss the measurement method used in our study in detail.

3.4.1. Measuring fault comprehension

To answer RQ-1 on the effects of explanations on the participants’ comprehension of the faults, we derived an average *comprehension score* based on a qualitative analysis of the plain text explanations provided by the participants for each of the three faults that were indicated by the tool. To avoid any bias, a scoring scheme was defined in advance, where individual participant responses could receive scores between 0 and 2. The scoring scheme is shown in Table 1.

Two of the authors scored all responses of the participants independently, and they did not know to which treatment or control group a particular response belonged to. On average, the two scorers largely agreed in terms of their assessments, leading to a high intraclass correlation coefficient. The researchers then inspected the explanations where they initially disagreed and then decided together on the final score.

We note that some participants in the HUMANEXPLANATION condition may have been tempted to copy and paste the provided (correct) explanation into their response, which would make it unclear if they truly understood the underlying problem. A manual inspection of the provided responses, however, revealed no case where the system’s response was simply copied. Instead, participants circumscribed the problem in their own words. We observed the same for the participants in the NATURALLANGUAGE group.

3.4.2. Measuring subjective perceptions

To assess the subjective quality perceptions of the different explanations (RQ-2), we designed a questionnaire that relied on instruments that were used earlier in the literature. First, we used the System Usability Scale (SUS) to assess the usability of the overall SMELLCHECKER tool. In addition, we took inspiration from the Technology Acceptance Model (TAM) (Davis, 1989) to assess the participants’ beliefs, attitudes, and behavioral intentions, both with respect to the explanations and to the tool as a whole. Specifically, we were interested in the system’s perceived *usefulness, ease-of-use, transparency and control, trust, satisfaction* and their *intentions to reuse or recommend* the system in the future. The detailed questionnaire items are shown in Table 3 below. In addition to these questions, we asked participants about demographics and their expertise with spreadsheets. Finally, participants could leave free-text comments before finalizing the questionnaire.

3.5. Study materials

In this section, we provide additional information with respect to materials used in our experiment.

Faulty spreadsheet. Selecting a suitable spreadsheet for the purpose of the study is a challenging task. On the one hand, the spreadsheet must not be too complex, because participants must be able to understand or at least properly guess the intended semantics of the calculations within a short period of time. On

Table 1
Scoring method for the explanations provided by the participants.

Faulty cell	Score	Rule
G33	0	Participant states that there is no fault or provides factually wrong explanation
	1	Participant states that a reference is missing
	2	Participant states that cell G31 is missing in the sum OR Participant states that the formula should be =SUM(G8:G31)
I25	0	Participant states that there is no fault or provides factually wrong explanation
	1	Participant states that the IF is not necessary OR Participant states that cell G25 should be referenced instead of cell G24
	2	Participants states that the formula should be consistent with the formulas in the same column OR Participants states that the formula should be =G25*H25
I34	0	Participant states that there is no fault or provides factually wrong explanation
	1	Participant states that a reference is missing
	2	Participant states that cell I8 is missing in the sum OR Participant states that the formula should be =SUM(I8:G31)

the other hand, a too simple spreadsheet might not be representative for real-world situations, see also for related discussions in Mukhtar et al. (2022). After a thorough analysis of existing spreadsheet corpora, we selected a real-world spreadsheet from the Enron Error corpus (Schmitz and Jannach, 2016) (see Fig. 2) because of its size, formula complexity, and the faults it contains.

The spreadsheet originally contained nine faulty formula cells. The cells in the range I25:I31 contained seven identical faults. The other cells with faulty formulas were G33 and I34. The SMELLCHECKER tool successfully located all faults of the spreadsheet. However, to keep the user study manageable, we corrected all but one of the identical faults, which left us with three faulty cells:

- A range error in the formula in cell G33, where we have =SUM(G8:G30) instead of =SUM(G8:G31),
- a double (range/logic) fault in cell I25, where we have =IF(H25;G24*H25;0) instead of =G25*H25, and
- another range error in the formula of cell I34, with =SUM(I9:I31) instead of =SUM(I8:I31).

Natural language explanations and post-task questionnaire. The exact phrasing of the natural language explanations (NATURALLANGUAGE) and the human explanations (HUMANEXPLANATION) for the three faulty cells of the spreadsheet used in the study are given in Table 2. Table 3 lists the exact questions that were asked to participants in the post-task questionnaire.

3.6. Participants

We recruited the participants for our study through the crowd-sourcing platform Prolific.⁷ Through an initial screening process, we only considered participants with a certain level of experience with spreadsheets, e.g., who could write formulas and manage data with spreadsheets. Furthermore, during the study, we asked the participants about their experience with spreadsheets. The statistics showed that more than half of the participants declared to have more than 7 years of experience with spreadsheets, more than half of the participants use spreadsheets daily, and less than 7% considered them as “beginners” concerning their experience. The detailed distribution of the participants’ responses in terms of “years of experience” are shown in Fig. 4. We believe the

participants have sufficient expertise in using spreadsheets and represent at least some common class of spreadsheet users in practice.

4. Results

In this section, we first analyze to what extent explanations can assist developers in understanding the faults of a spreadsheet (Section 4.1 on RQ-1) and then study the subjective quality perceptions of the participants in more depth (Section 4.2 on RQ-2).

4.1. Fault comprehension (RQ-1)

Fault comprehension scores. We recall that we assess the fault comprehension level of participants through a manual inspection and scoring procedure by two independent assessors. The outcomes of this scoring process for each faulty cell and participant group are shown in Fig. 5.

We observe that the general pattern is consistent across the three faulty cells. The highest level of fault comprehension was achieved by the participants of the HUMANEXPLANATION group. This is expected, as the explanations for this group were created manually and should serve as an upper bound in terms of fault comprehension. The average comprehension scores on our 0–2 scale for the three cells ranged from 1.78 to 1.90, with an average of 1.83. The lowest score of 1.78 was achieved for cell I25, which contained the most complex fault.

The second highest comprehension level was achieved by participants in the NATURALLANGUAGE group, with an average of 1.58 across all faults. Looking at the detailed scoring results, we found that participants in this group fully comprehended (and properly described) more than half of the faulty cells, and they showed a reasonable understanding of the problems in the other cells.

Interestingly, the visualization-based explanations, which are based on LIME outputs like NATURALLANGUAGE, did not work too well in our study. In fact, on average the fault comprehension level for this treatment group was even slightly lower than for the BLACKBOX group, where participants were only shown the overall suspiciousness score. This may indicate that the chosen visualization was too difficult to interpret for the participants, or they did not contain sufficient information for the participants. On an absolute scale, the average comprehension level was at 1.4 for the BLACKBOX explanations, and 1.12 for the VISUALIZATION approach. Again, the lowest scores were obtained for the most

⁷ <https://www.prolific.co/>, participants were paid 7€ for participating in the study.

Table 2
Top three product metrics for faulty cells.

Faulty cell	Predictive product metric	Natural language explanation	Human explanation
G33	Missing neighbor cells (top/left)	References to cells that are to the left or above this cell may be missing.	The reference to cell G31 is missing in this formula.
	References to cells	There is an unusual number of references to other cells.	
	Number of cells in calculation	There is an unusual number of references in this formula.	
I25	IF formula complexity	The IF statement is either not required or it is too complex.	The IF statement is not required in this formula and the reference to cell G24 is incorrect.
	Same type cell distance (row)	This row contains both formula cells and data cells.	
	References from other cells	There is an unusual number of references to this cell.	
I34	References to cells	There is an unusual number of references to other cells.	The reference to cell I8 is missing in this formula.
	Missing neighbor cells (top/left)	References to cells that are to the left or above this cell may be missing.	
	Number of cells in calculation	There is an unusual number of references in this formula.	

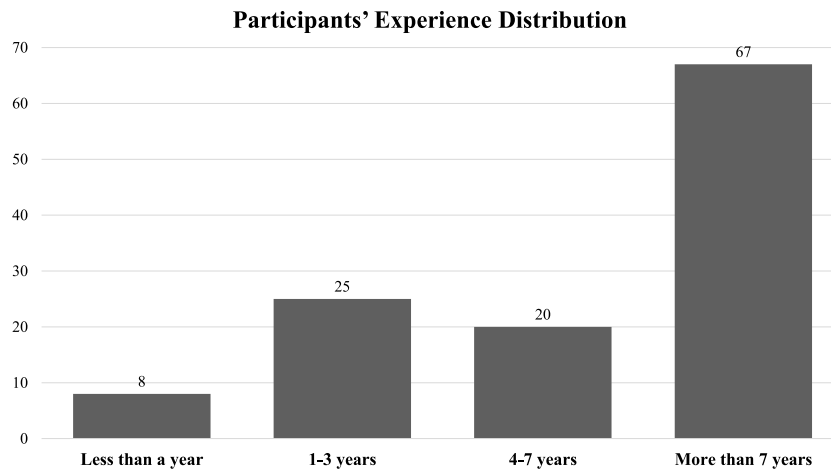


Fig. 4. Distribution of years of experience of the participants.

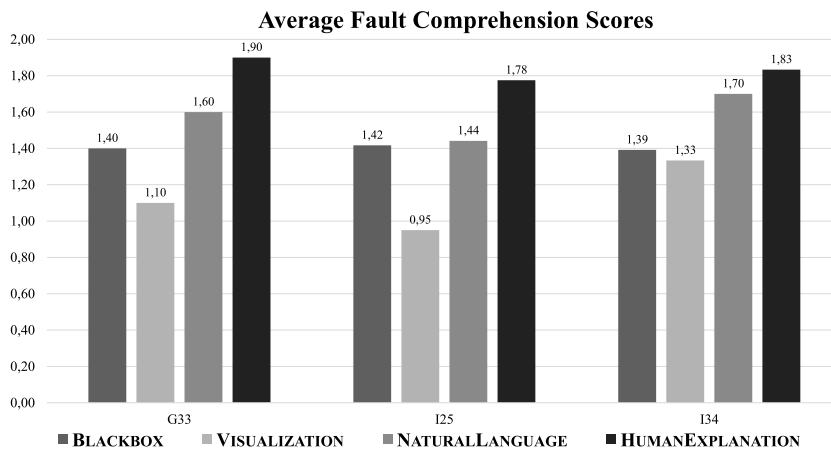


Fig. 5. Average fault comprehension score.

complex fault in each group. Overall, the higher average score for the BLACKBOX explanations might indicate that the visualization may have in fact distracted the participants, and that the participants in the BLACKBOX group could better focus on understanding the problems in the faulty cells. Looking at individual responses by participants in the VISUALIZATION group, we could observe that they in a number of cases interpreted the feature

importance values as shown in the visualizations in a wrong way. The NATURALLANGUAGE explanation approach, in contrast, seems to be indeed helpful to participants in terms of fault comprehension.

The distribution of the comprehension scores for the individual cells and participant groups are shown in Fig. 6 in the form of box plots. An ANOVA analysis revealed significance differences

Table 3
Post-Task Questionnaire Items.

ID	Question and answer options
Demographics	
D1	How often do you use spreadsheets with formulas in your daily work? <i>Daily/Once in a week/Once in a month/Rarely/Don't use at all</i>
D2	How many years have you been using spreadsheets in your daily work? <i>Less than a year/1–3 years/4–7 years/More than 7 years</i>
D3	I rate my expertise with spreadsheets as follows: <i>Beginner/Intermediate/Advanced/Expert</i>
D4	How old are you? <i>18–24 years/25–29 years/30–34 years/35–39 years/40 and over</i>
D5	What is your gender? <i>Male/Female/Prefer not to say</i>
<i>All answers are on a 1–5 Likert scale (1=strongly disagree; 5=strongly agree)</i>	
User Beliefs: Perceived Usefulness	
U1	The explanations provided by SmellChecker helped me understand why the marked cells were suspicious.
U2	I could not understand the explanations provided by SmellChecker.
U3	The explanations provided by SmellChecker were distracting.
U4	The explanations provided by SmellChecker allowed me to accomplish the task more quickly.
User Beliefs: Perceived Ease of Use	
E1	I would need an expert opinion to understand the explanations provided by SmellChecker.
E2	The interaction with SmellChecker was clear and understandable.
E3	The explanations provided by SmellChecker were easy to understand.
E4	It took too much effort to understand the explanations provided by SmellChecker.
User Beliefs: Control	
C1	I felt in control of the various functionalities of SmellChecker.
User Beliefs: Transparency	
T1	I understand why the cells were marked suspicious through the explanations.
<i>All answers are on a 1–5 Likert scale (1=strongly disagree; 5=strongly agree)</i>	
User Attitudes: Trust and Confidence	
TC1	The explanations provided by SmellChecker are trustworthy.
TC2	Overall, I trust SmellChecker.
User Attitudes: Satisfaction	
S1	I needed to learn a lot of things before I could get going with SmellChecker.
S2	My overall satisfaction with SmellChecker is high.
<i>All answers are on a 1–5 Likert scale (1=strongly disagree; 5=strongly agree)</i>	
Behavioral Intentions: Recommendation Intentions	
R1	I would recommend SmellChecker to my friends or colleagues who use spreadsheet tools.
Behavioral Intentions: Intention to use	
I1	I would like to use SmellChecker again because I found the explanations helpful.
I2	I could not understand the explanations, so I would not use SmellChecker again.
<i>All answers are on a 1–5 Likert scale (1=strongly disagree; 5=strongly agree)</i>	
System Usability	
SU1	I would like to use SmellChecker frequently.
SU2	I found SmellChecker unnecessarily complex.
SU3	I thought SmellChecker was easy to use.
SU4	I would need the support of a technical person to be able to use SmellChecker.
SU5	I found the various functions in SmellChecker were well integrated.
SU6	There was too much inconsistency in SmellChecker.
SU7	I would imagine that most people would learn to use SmellChecker very quickly.
SU8	I found SmellChecker very cumbersome to use.
SU9	I felt very confident using SmellChecker.
SU10	I needed to learn a lot of things before I could get going with SmellChecker.
<i>All answers are free text</i>	
Comments and Suggestions	
CS1	What do you think about the explanation(s) given for the suspicious cells?
CS2	Write your comments and suggestions about SmellChecker in general here.

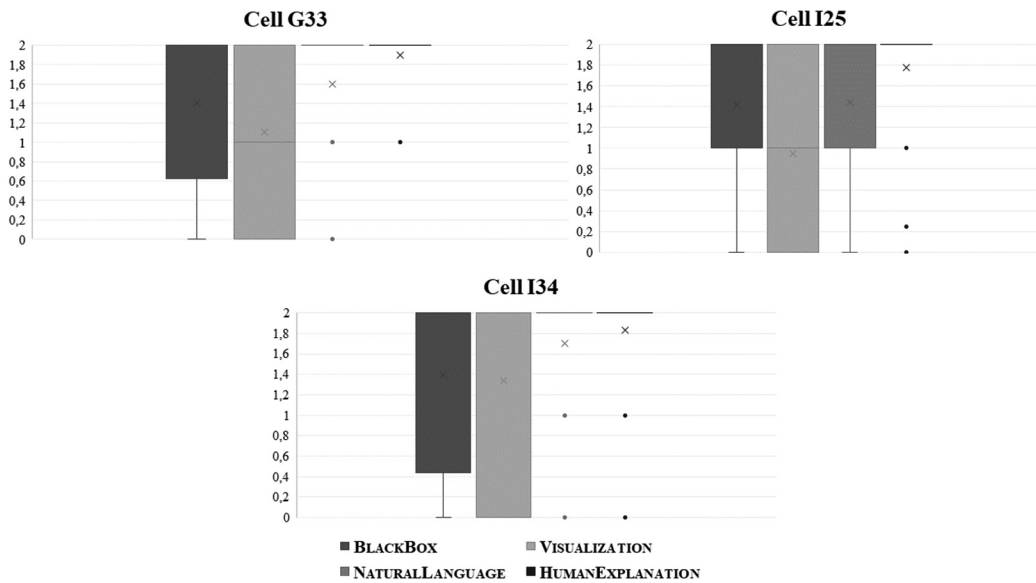


Fig. 6. Box plot (Fault comprehension score).

Table 4

Overview of significant differences in terms of fault comprehension.

G33	I25	I34
HUMANEXPLANATION > BLACKBOX **	–	–
HUMANEXPLANATION > VISUALIZATION **	HUMANEXPLANATION > VISUALIZATION **	HUMANEXPLANATION > VISUALIZATION *
–	NATURALLANGUAGE > VISUALIZATION **	–

**Indicates significance at $\alpha=0.05$.

*Indicates significance at $\alpha=0.1$.

Table 5

Concepts Categorization.

Sentiment	Concepts
Positive	Useful
	Helpful
	Understandable
	Good/Great
	Recommend
Negative	Complicated/Cumbersome
	Dislike
	Confusing
	Unhelpful
	Needs improvement
Neutral	General remarks

between the participant groups for each cell.⁸ We, therefore, applied Tukey's HSD post-hoc tests with Bonferroni correction.⁹ The tests revealed a number of significant differences between the participant groups, which we summarize in Table 4.

The extent of the differences in the mean comprehension scores vary across the faulty formulas. The difference between the "optimal" human explanations and the visualization-based approach are at least moderately significant in all cases. In the case of the most complex fault in cell I25, it turned out that also the automatically generated NATURALLANGUAGE approach was significantly better than the VISUALIZATION technique.

Overall, answering RQ-1, we see our results as *clear indications that well-designed explanations can be a suitable tool for improved*

fault comprehension in the spreadsheet debugging process. It also turns out that the usefulness of explanations depends on their particular implementation and there are indications that it may also depend on the complexity and nature of the faults. In our specific comparison, we found that showing natural language descriptions for important predictor variables in the NATURALLANGUAGE approach is promising. A widely used visualization approach (VISUALIZATION), on the other hand, actually can even have slightly detrimental effects compared to a situation where the system does not provide specific explanations at all.

Feedback analysis. Given the observations regarding the participants' level of fault comprehension as just discussed, we examined if the free-form feedback on the explanations – from questionnaire item CS1¹⁰ in Table 3 – aligns with these general tendencies. To analyze this aspect, we used Inductive Coding (Chandra and Shang, 2019) as a technique to extract concepts or themes from a body of text in a structured way. During this process, the coders were not aware to which participant group a given free text belonged to. The identified concepts are shown in Table 5. To obtain a broader picture of the general assessment of the participants of the explanations we then further categorized the concept as having either a positive, negative, or neutral sentiment.

We then counted the occurrence of each concept in the free-form feedback text. The frequency distribution of the concept sentiments for the groups is presented in Fig. 7.

We clearly observe that the HUMANEXPLANATION and NATURALLANGUAGE groups provided largely positive feedback on the explanations. The participants who saw the visualization-based explanations were less positive, but still provided positive feedback more often than the participants who only saw the suspiciousness scores in the BLACKBOX group. However, participants

⁸ The p-values were below 0.001 for cell G33 and cell I25, and at 0.03 for cell I34.

⁹ Bonferroni correction is done to account for the multiple comparisons. After applying the correction, the α level to reject the null hypothesis is lowered from 0.05 to 0.0083. An α level of 0.1 is lowered to 0.016.

¹⁰ The exact question read: "What do you think about the explanation(s) given for the suspicious cells?"

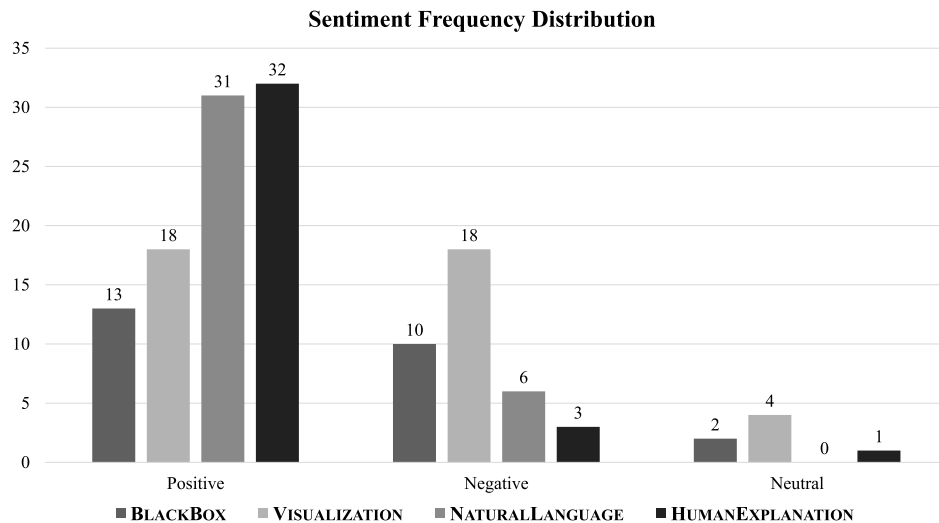


Fig. 7. Sentiment frequency distribution.

in the VISUALIZATION group also frequently reported negative feedback. This supports our observations from the fault comprehension analysis that participants often may have had difficulties in interpreting the visualization.¹¹ Overall, this qualitative analysis of the free-form feedback seems largely consistent with the fault comprehension analyses reported above, where natural language explanations generally had a positive effect.

4.2. Subjective perceptions (RQ-2)

We recall that the study participants were asked to answer in-depth questionnaires after completing the main task of analyzing potential faults. We evaluated four dimensions of perceptions, namely *Tool Usability*, *User Attitudes*, *User Beliefs*, and *Behavioral Intentions*. Our analyses involve a larger number of comparisons and significance tests, and we share all detailed outcomes in an online repository.¹² For the sake of brevity, we limit our discussions here to the general trends and insights gathered through our user study.

Tool usability. Participants answered a 10-item questionnaire from the System Usability Scale (SUS), using a 1-to-5 scale. The range of the overall SUS score for a system lies between 0 and 100, which is obtained by normalizing the response values of the questionnaire items. For this purpose, value 1 is subtracted from each positively framed question's score, and each negatively framed question's score is subtracted from value 5. Finally, the sum of these scores is multiplied by 2.5 to normalize the scores between 0 and 100. The overall SUS scores for the different participant groups are shown in Table 6.

Overall, we observe that all versions of the tool obtained overall scores that are close or markedly higher than the general threshold of 68. Systems that score higher than this value are usually considered above average or "good". The scores that were obtained for the tool versions that provided HUMANEXPLANATION and NATURALLANGUAGE explanations are much higher than for the other types of explanations. This is in line with our findings reported above with respect to fault comprehension, and

Table 6

Overall System Usability Scale (SUS) scores. Values larger than 68 are considered above average ("good").

Treatment Group	SUS score
HUMANEXPLANATION	82.25
NATURALLANGUAGE	79.50
VISUALIZATION	68.03
BLACKBOX	67.26

these results provide strong indications that these two types of explanations can strongly contribute to the usability of the SMELLCHECKER tool as a whole. We acknowledge that the observed differences may be particularly pronounced in our study, where the participants had to verbalize the potential reasons for a suspected fault as a main task when interacting with the tool.

The mean responses for each of the items of the SUS scale are shown in Fig. 8. Aligned with the overall scores, we find that the assessments by participants in the HUMANEXPLANATION and NATURALLANGUAGE groups were consistently better than those for the other two participant groups. While in some cases the differences are not too large, there are stronger differences for some of the questions. Looking at item **SU2**, for example, we observe that the VISUALIZATION group found the SMELLCHECKER tool markedly more complex than the other groups. This effect can apparently be attributed to the design of the explanations, because nothing else was changed across the groups.

User beliefs. The average responses to the questionnaire items relating to *user beliefs* are presented in Fig. 9.¹³ In general, the participants in the HUMANEXPLANATION group reported the best values for various aspects covered in this questionnaire, followed by the participants in the NATURALLANGUAGE group. Looking at some of the details, we for example find that participants in the HUMANEXPLANATION and NATURALLANGUAGE groups found the explanations to be more *useful*, more *understandable* and less *distracting* than participants in the other groups. Conversely, participants in the VISUALIZATION group often seemed to have difficulties understanding the explanations. Again, this is aligned with the other observations that we reported so far.

¹¹ Fig. 7 shows a small set of negative feedback also for the HUMANEXPLANATION condition. An inspection of these cases revealed that all but one of the few negative comments did not refer to the explanations, but to other aspects like the complexity of the installation of the plug-in. The one remaining comment with a negative sentiment mentioned that "a step-by-step guide, particularly on I25, would have been useful".

¹² <https://doi.org/10.5281/zenodo.7703947>

¹³ The mean participant responses and standard deviations for user beliefs, attitudes, and behavioral intentions can also be found in Table 7 in Appendix.

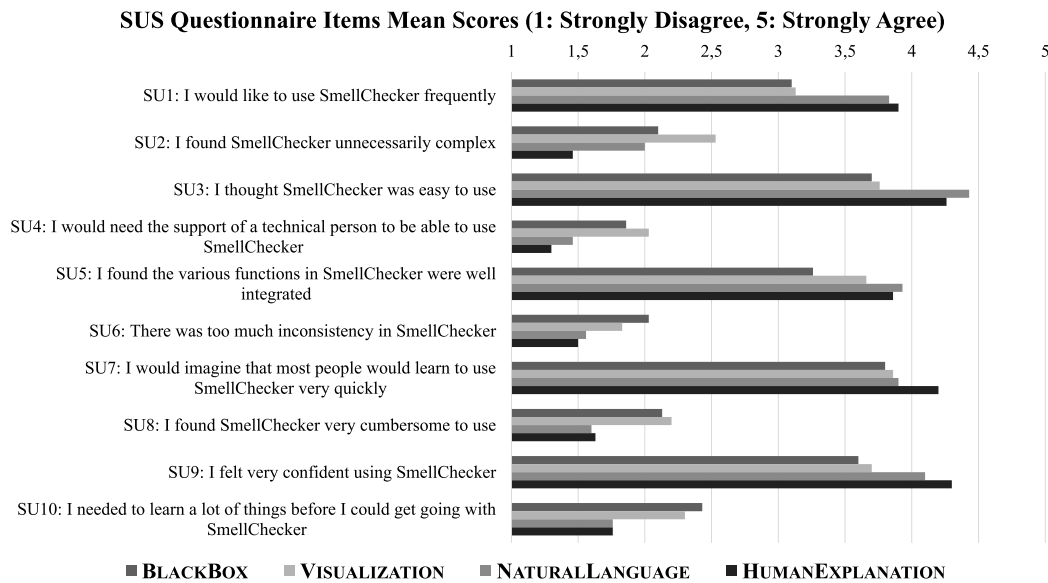


Fig. 8. Mean scores of the SUS questionnaire items.

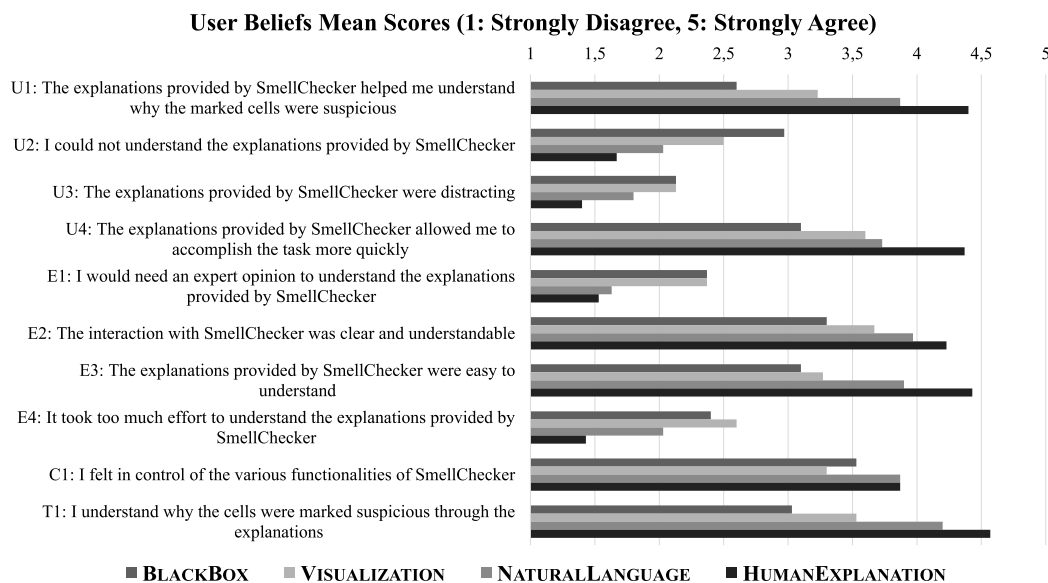


Fig. 9. User beliefs: Perceived usefulness (U1–U4), perceived ease of use (E1–E4), control (C1), transparency (T1).

An interesting observation can be made of item *T1*, where participants had to inform to what extent they understood why the tool marked a cell as being potentially faulty. While the participants in the BLACKBOX group gave lower scores than participants in other groups, their average response was not at the very low end. This is to some extent unexpected because the tool in that condition did not provide any information about the possible reasons for a cell being suspiciousness other than an overall suspiciousness score. Thus, we suspect that participants in the BLACKBOX group may have interpreted the question at a different level. Their reasoning might have been that they understood that the cell was marked because it had a high suspiciousness score, i.e., they might not have considered reasons that relate to the formula in the cell itself.

User attitudes. In terms of user attitudes, we asked participants about their trust and confidence¹⁴ with respect to the explanations and about their overall satisfaction. The results for this part of the questionnaire are shown in Fig. 10.

We find that participants in the HUMANEXPLANATION group consistently reported the best values in terms of these factors,

¹⁴ We acknowledge that in general trust towards a software application is a complex and multi-faceted concept, and may be based on various factors such as multiple positive past experiences with a given tool or similar tools over time, or from the credibility of the provider of the application. In our study, we assess the participants' trust (or: perceived trustworthiness of the system) after one single experience. This may represent a certain limitation of our present study and earlier works in this area; see Wu et al. (2011) for a meta-analysis of the impact of trust in the Technology Acceptance Model.

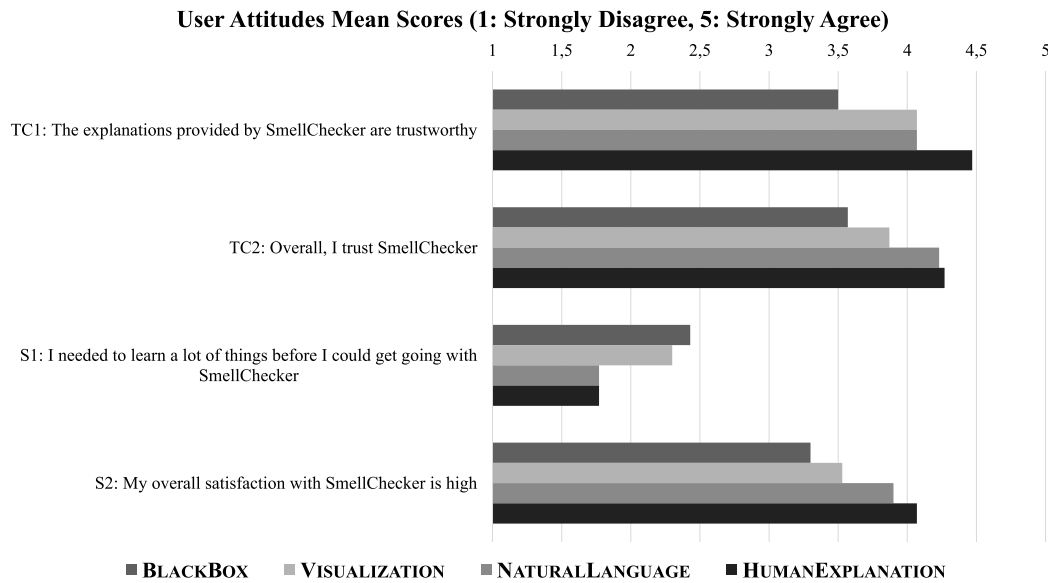


Fig. 10. User attitudes: Trust and confidence (TC1–TC2), satisfaction (S1–S2).

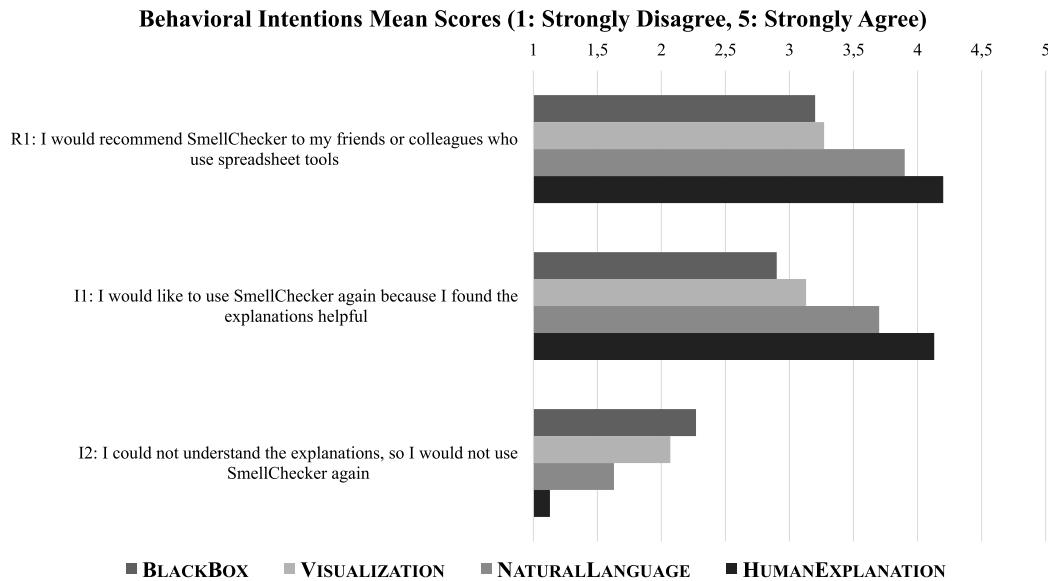


Fig. 11. Behavioral intentions: Recommendation intentions (R1), intentions to use (I1, I2).

i.e., the participants in the HUMANEXPLANATION group found SMELLCHECKER and the explanations more trustworthy and their overall satisfaction is slightly higher compared to the other groups. The average responses of the NATURALLANGUAGE group for trustworthiness and satisfaction were also encouraging, e.g., they are around 4 for the positively framed question items (TC1, TC2 and S2). The lowest values were consistently observed for the BLACKBOX group. On an absolute scale, these values were also not too bad, and participants in this group were still satisfied to a certain extent with the overall tool.

An interesting observation can be made with respect to the VISUALIZATION group. Even though the level of fault comprehension was rather low for this group, and even though participants found these explanations to be rather complex and more difficult to understand (see *User Beliefs*), they reported the same level of trust as the participants in the NATURALLANGUAGE group. This may have been caused by the fact that participants in these groups were provided information about the same set of features, albeit in a different form. We may also speculate that the more formal,

numeric explanation in the VISUALIZATION group may have led participants to attribute similar levels of trust, even though the visualization was more complex and less understandable than the NATURALLANGUAGE explanations.

Behavioral intentions. We report the responses of the questionnaire items relating to *behavioral intentions* (to reuse the tool and to recommend it to others) in Fig. 11.

The observations so far made in terms of user beliefs and user attitudes are clearly reflected in the responses to these final questions. Participants in the HUMANEXPLANATION condition expressed the highest level of intentions to use the tool in the future and to recommend it. The NATURALLANGUAGE explanations led to slightly lower positive intentions, followed by the VISUALIZATION and the BLACKBOX explanations. Overall, we observe a positive effect of adding understandable explanations both in terms to the tool as a whole (R1) and in terms of the contribution of the explanations to the future intentions (I1, I2).

Overall, answering **RQ-2**, our results indicate that *well-designed explanations can help to improve various quality perceptions and the trust towards a spreadsheet debugging tool*.

5. Implications & threats to validity

5.1. Implications

Our research has a number of important practical implications. With respect to **RQ-1**, the main insight of our study is that providing explanations in the context of automated fault prediction support can indeed help spreadsheet users improve their understanding of the potential faults that are highlighted by such a tool. Thus, designers of fault prediction tools should consider enhancing their tools by a corresponding functionality, since an increased level of fault comprehension will naturally be beneficial for the subsequent fault removal step.

Second, and also importantly, we found that the design of the explanation matters a lot. The most effective explanations in our study were manually engineered ones written in natural language, which we considered as an upper bound in our experiments. The natural language explanations, which were automatically selected based on the widely-used LIME approach, however, turned out to be promising as well. The approach requires some initial manual effort, but this effort is very limited as it amounts to merely creating one natural language sentence for each predictor variable (feature) of the underlying supervised learning problem. We recall that these descriptions are not tied to a specific spreadsheet or application domain. Thus, these explanations can be crafted once and then shipped with the debugging tool. In the context of design choices, we furthermore found that some forms of explanations might not be effective at all or even have a slightly detrimental effect on fault comprehension. This may happen when the explanation is not easy to comprehend. In our study, we found indications that a popular form of visualizations in the literate based on bar charts may even misguide spreadsheet users.

Given these observations in the context of **RQ-1**, we conclude that any form of explanations that are presented to users must be carefully evaluated in terms of their true usefulness with studies involving humans. In our case, this evaluation was done in two ways: (a) by manually scoring the participants' level of fault comprehension in a quantitative way and (b) by qualitatively analyzing the participants' sentiment towards the provided explanations as expressed in the free-form feedback statements. The observations for both types of analysis were well aligned in our study. Generally, relying on more than one form of evaluation procedure is advisable in such forms of human-centered research to obtain robust results.

Our analyses in the context of **RQ-2** highlight that explanations in the spreadsheet domain may not only serve the primary target of improving the participants' understanding of potential faults. We found that well-designed explanations lead to better quality perceptions in terms of usability and usefulness, and that such explanations may help to increase the users' trust in the debugging system as a whole and, consequently, their intentions to use the tool or to recommend it to others. These findings underline that there are several potential advantages of providing explanations when highlighting parts of a program to be as potentially faulty.

5.2. Threats to validity

Our work comes not without limitations. First, our study was based on one particular spreadsheet and on one given set of

faults.¹⁵ Thus, it remains to study if we would observe similar effects for spreadsheets that are largely different from the one used in our experiment. Nonetheless, we believe that the spreadsheet used in our study and the faults it contains are representative of a larger set of spreadsheets that can be found in practice. The used spreadsheet in fact is a real-world one and all the faults in it were also "natural". Also, in terms of the number and complexity of the formulas, there are many spreadsheets in the Enron error corpus (Fisher and Rothermel, 2005) of real-world spreadsheets that are very similar to our spreadsheet.

Generally, our work shares the potential limitations of any user study in which study participants are completing tasks in an artificial environment and setting. In terms of the *realism* of the study, we are confident that our setting was perceived to be quite natural by participants, given their feedback on the general usefulness of the tool and their intentions to use such a tool in the future. In terms of study participants, we relied on crowdworkers. Since most of these crowdworkers indicated that they had at least seven years of experience with spreadsheets, we argue that the study participants are representative for at least some part of the typical population of spreadsheet users in practice. Furthermore, to ensure that our results are not based on the work of potentially inattentive and unmotivated crowdworkers, we (a) only admitted crowdworkers with a positive track record to the study and (b) implemented attention checks to only include the responses of careful crowdworkers in our analyses.

Furthermore, our study was so far limited to four conditions, where in two of them the explanations were automatically selected and/or visualized based on a widely-used technique from the field of explainable AI. Other forms of explaining the fault predictions are certainly possible and are part of our future work. However, we recall here that the goal of our study was not to find the "best" form of explanations but to take the first step towards an understanding of the general usefulness of providing explanations for automated fault predictions. Moreover, in our experiment, we assumed that there were no "false positives", i.e., that the tool only marked formulas as suspicious, which were faulty. While avoiding such false alarms is generally a problem of the underlying fault prediction method, it may be interesting to study how different forms of explanations affect spreadsheet users when there is no fault. For example, one hypothesis could be that more understandable explanations are also helpful for users to detect that the highlighted cell is correct. But, it may also be that users overly rely on the predictions and explanations by the tool and may even start modifying a correct formula, see Mukhtar et al. (2022) for a study on tool over-reliance in the context of spreadsheets.

6. Summary and outlook

In a widely cited study, Parnin and Orso (2011) questioned if certain automated debugging tools from the literature are *really* helping developers. They concluded that this might not always be the case and that this may be caused by certain assumptions that are made by researchers which may not hold in practice, e.g., how developers inspect lists of potentially faulty program elements. As a consequence, the authors call for more research that study questions of *fault comprehension* and, therefore, for more research that is based on studies with users.

In our research, we have addressed the question of the usefulness of a recent spreadsheet fault prediction technique in this spirit for the class of spreadsheet programs. In particular, we have investigated to what extent it may be beneficial to

¹⁵ Also, certain types of faults, e.g., when users entered a constant value instead of a formula in a cell, were not investigated yet in our present study.

Table 7

Mean (standard deviations) of participant responses in terms of user beliefs, user attitudes, and behavioral intentions. The exact questions are shown in Table 3.

		Question ID	BLACKBOX	VISUALIZATION	Natural Language	Human Explanation
User Beliefs	Perceived Usefulness	U1	2.60 (1.33)	3.23 (1.05)	3.87 (1.02)	4.40 (0.88)
		U2	2.97 (1.40)	2.50 (1.38)	2.03 (1.14)	1.67 (1.04)
		U3	2.13 (1.18)	2.13 (1.18)	1.80 (0.98)	1.40 (0.92)
		U4	3.10 (1.42)	3.60 (1.31)	3.73 (1.29)	4.37 (0.80)
	Perceived Ease of Use	E1	2.37 (1.30)	2.37 (1.38)	1.63 (0.95)	1.53 (0.92)
		E2	3.30 (1.19)	3.67 (1.14)	3.97 (0.87)	4.23 (0.80)
		E3	3.10 (1.27)	3.27 (1.15)	3.90 (0.91)	4.43 (0.72)
		E4	2.40 (1.14)	2.60 (1.20)	2.03 (1.02)	1.43 (0.76)
	Control	C1	3.53 (1.15)	3.30 (1.10)	3.87 (1.02)	3.87 (0.85)
	Transparency	T1	3.03 (1.40)	3.53 (1.33)	4.20 (0.83)	4.57 (0.56)
User Attitudes	Trust and Confidence	TC1	3.50 (1.18)	4.07 (0.96)	4.07 (0.93)	4.47 (0.72)
		TC2	3.57 (1.09)	3.87 (1.09)	4.23 (0.84)	4.27 (0.73)
	Satisfaction	S1	2.43 (1.41)	2.30 (1.27)	1.77 (1.02)	1.77 (0.96)
		S2	3.30 (1.22)	3.53 (0.99)	3.90 (0.83)	4.07 (0.73)
Behavioral Intentions	Recommendation Intentions	R1	3.20 (1.28)	3.27 (1.26)	3.90 (1.16)	4.2 (0.87)
		I1	2.90 (1.33)	3.13 (1.36)	3.70 (1.07)	4.13 (1.02)
	Intention to use	I2	2.27 (1.21)	2.07 (1.18)	1.63 (0.80)	1.13 (0.34)

explain the tool's fault prediction to users. Overall, we find strong indications that providing explanations for the outcomes of an underlying machine learning model can indeed improve the usefulness and quality perception of the fault localization tool. To our knowledge, this is the first work in the area of spreadsheet debugging, and we found also very limited works that address the role of explanations for fault prediction techniques for general software. More research is therefore required to understand if the insights from our experiment in the spreadsheet domain generalize for other types of software programs. Nonetheless, in a broader perspective, we see our work also a contribution to the area of explainable AI, which follows the assumption that a wider and adoption of AI-fueled technology might often depend on a system's capability of explaining its decisions or recommendations.

In terms of future works, we plan to investigate alternative ways of explaining the system's fault predictions to users, including in particular other forms of visualizations, given that visualizations have proven a promising means to support spreadsheet debugging processes in the past, see Jannach et al. (2014). Moreover, besides validating our findings with other spreadsheets and to analyze to what extent the effectiveness of explanations is correlated with the complexity of the faults, it remains important to study the usefulness of explanations when alternative tools and underlying techniques are used for fault prediction, e.g., based on spectrum-based fault localization or on model-based reasoning.

CRediT authorship contribution statement

Adil Mukhtar: Conceptualization, Methodology, Software, Formal analysis, Investigation, Resources, Data curation, Visualization, Writing – original draft, Writing – review & editing. **Birgit Hofer:** Software, Validation, Formal analysis, Writing – review & editing, Visualization. **Dietmar Jannach:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing, Supervision, Funding acquisition. **Franz Wotawa:** Conceptualization, Methodology, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We share a link to the data in the manuscript.

Acknowledgments

The work described in this paper is funded by the Austrian Science Fund (FWF) project Interactive Spreadsheet Debugging under contract number P 32445.

Appendix

See Table 7.

References

- Ahmed, M.R., Ali, M.A., Ahmed, N., Zamal, M.F.B., Shamrat, F.J.M., 2020. The impact of software fault prediction in real-world application: An automated approach for software engineering. In: Proceedings of 2020 the 6th International Conference on Computing and Data Engineering. Association for Computing Machinery, pp. 247–251.
- Ang, A., Perez, A., Van Deursen, A., Abreu, R., 2017. Revisiting the practical use of automated software fault localization techniques. In: 2017 IEEE International Symposium on Software Reliability Engineering Workshops. ISSREW, pp. 175–182.
- Aurigemma, S., Panko, R.R., 2010. The detection of human spreadsheet errors by humans versus inspection (auditing) software. In: EuSpRIG 2010 Conference. pp. 1–14.
- Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-Lopez, S., Molina, D., Benjamins, R., Chatila, R., Herrera, F., 2020. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. Inf. Fusion 58, 82–115.
- Burg, B., Bailey, R., Ko, A.J., Ernst, M.D., 2013. Interactive record/replay for web application debugging. In: Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology. UIST '13, pp. 473–484.
- Catal, C., 2011. Software fault prediction: A literature review and current trends. Expert Syst. Appl. 38 (4), 4626–4636.
- Chandra, Y., Shang, L., 2019. Inductive coding. In: Chandra, Y., Shang, L. (Eds.), Qualitative Research using R: A Systematic Approach. Springer, pp. 91–106.
- Cheung, S.-C., Chen, W., Liu, Y., Xu, C., 2016. CUSTODES: automatic spreadsheet cell clustering and smell detection using strong and weak features. In: 38th International Conference on Software Engineering. pp. 464–475.
- Choudhary, G.R., Kumar, S., Kumar, K., Mishra, A., Catal, C., 2018. Empirical analysis of change metrics for software fault prediction. Comput. Electr. Eng. 67, 15–24.
- Czibula, G., Marian, Z., Czibula, I.G., 2014. Software defect prediction using relational association rule mining. Inform. Sci. 264, 260–278, Serious Games.
- Davis, F.D., 1989. Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Q. 13 (3), 319–340.

- Dou, W., Cheung, S., Wei, J., 2014. Is spreadsheet ambiguity harmful? Detecting and repairing spreadsheet smells due to ambiguous computation. In: 36th International Conference on Software Engineering. pp. 848–858.
- Ducasse, M., 1993. A pragmatic survey of automated debugging. In: International Workshop on Automated and Algorithmic Debugging. AADeBUG, In: Lecture Notes in Computer Science (LNCS), vol. 749, Springer, pp. 1–15.
- Fey, G., Sulflow, A., Drechsler, R., 2010. Towards unifying localization and explanation for automated debugging. In: 2010 11th International Workshop on Microprocessor Test and Verification. pp. 3–8.
- Fisher, II, M., Rothermel, G., 2005. The EUSES spreadsheet corpus: a shared resource for supporting experimentation with spreadsheet dependability mechanisms. *ACM SIGSOFT Softw. Eng. Notes* 30 (4), 1–5.
- Groce, A., Chaki, S., Kroening, D., Strichman, O., 2006. Error explanation with distance metrics. *Int. J. Softw. Tools Technol. Transf.* 8, 229–247.
- He, P., Li, B., Liu, X., Chen, J., Ma, Y., 2015. An empirical study on software defect prediction with a simplified metric set. *Inf. Softw. Technol.* 59, 170–190.
- Hermans, F., Pinzger, M., van Deursen, A., 2012. Detecting and visualizing inter-worksheet smells in spreadsheets. In: 34th International Conference on Software Engineering. pp. 441–451.
- Hofer, B., Jannach, D., Koch, P., Schekotihin, K., Wotawa, F., 2021. Product metrics for spreadsheets – A systematic review. *J. Syst. Softw.* 175, 110910.
- Jannach, D., Schmitz, T., Hofer, B., Schekotihin, K., Koch, P.W., Wotawa, F., 2019. Fragment-based spreadsheet debugging. *Autom. Softw. Eng.* 26 (1), 203–239.
- Jannach, D., Schmitz, T., Hofer, B., Wotawa, F., 2014. Avoiding, finding and fixing spreadsheet errors – A survey of automated approaches for spreadsheet QA. *J. Syst. Softw.* 94, 129–150.
- Ko, A.J., Myers, B.A., 2004. Designing the whyline: a debugging interface for asking questions about program behavior. In: CHI. ACM, pp. 151–158.
- Ko, A.J., Myers, B.A., 2008. Debugging reinvented: Asking and answering why and why not questions about program behavior. In: Proceedings of the 30th International Conference on Software Engineering. ICSE '08, pp. 301–310.
- Koch, P., Schekotihin, K., Jannach, D., Hofer, B., Wotawa, F., 2019. Metric-based fault prediction for spreadsheets. *IEEE Trans. Softw. Eng.* 1.
- Koch, P., Schekotihin, K., Jannach, D., Hofer, B., Wotawa, F., 2019. Metric-based fault prediction for spreadsheets. *IEEE Trans. Softw. Eng.* 1.
- Li, Z., Jing, X.-Y., Zhu, X., 2018. Progress on approaches to software defect prediction. *IET Softw.* 12 (3), 161–175.
- Li, N., Shepperd, M., Guo, Y., 2020. A systematic review of unsupervised learning techniques for software defect prediction. *Inf. Softw. Technol.* 122, 106287.
- Lundberg, S., Lee, S.-I., 2017. A unified approach to interpreting model predictions. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. NIPS '17, pp. 4768–4777.
- Malhotra, R., 2015. A systematic review of machine learning techniques for software fault prediction. *Appl. Soft Comput.* 27, 504–518.
- Manjula, C., Florence, L., 2019. Deep neural network based hybrid approach for software defect prediction using software metrics. *Cluster Comput.* 22, 9847–9863.
- Matloob, F., Ghazal, T.M., Taleb, N., Aftab, S., Ahmad, M., Khan, M.A., Abbas, S., Soomro, T.R., 2021. Software defect prediction using ensemble learning: A systematic literature review. *IEEE Access* 9, 98754–98771.
- Miller, T., 2019. Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence* 267, 1–38.
- Miryeganeh, N., Hashtroudi, S., Hemmati, H., 2021. GloBug: Using global data in fault localization. *J. Syst. Softw.* 177, 110961.
- Mukhtar, A., Hofer, B., Jannach, D., Wotawa, F., 2022. Spreadsheet debugging: The perils of tool over-reliance. *J. Syst. Softw.* 184, 111119.
- Murray, W.R., 1988. Automatic Program Debugging for Intelligent Tutoring Systems.
- Nunes, I., Jannach, D., 2017. A systematic review and taxonomy of explanations in decision support and recommender systems. *User Model. User-Adapt. Interact.* 27 (3–5), 393–444.
- Okutan, A., Yildiz, O., 2014. Software defect prediction using Bayesian networks. *Empir. Softw. Eng.* 19, 154–181.
- Omri, S., Sinz, C., 2020. Deep learning for software defect prediction: A survey. In: Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops. pp. 209–214.
- Panko, R.R., 1998. What we know about spreadsheet errors. *J. Organ. End User Comput.* 10 (2), 15–21.
- Panko, R.R., 2008. Revisiting the Panko–Halverson taxonomy of spreadsheet errors. In: Proc. European Spreadsheet Risks Int. Grp. (EuSpRIG) 2008. pp. 199–220.
- Parnin, C., Orso, A., 2011. Are automated debugging techniques actually helping programmers? In: 2011 International Symposium on Software Testing and Analysis. pp. 199–209.
- Radjenović, D., Heričko, M., Torkar, R., Živković, A., 2013. Software fault prediction metrics: A systematic literature review. *Inf. Softw. Technol.* 55 (8), 1397–1418.
- Rathore, S., Kumar, S., 2019. A study on software fault prediction techniques. *Artif. Intell. Rev.* 51, 255–327.
- Ribeiro, M., Singh, S., Guestrin, C., 2016. “Why should I trust you?”: Explaining the predictions of any classifier. In: Proceedings NAACL 2016. pp. 97–101.
- Ruthruff, J.R., Burnett, M., Rothermel, G., 2006. Interactive fault localization techniques in a spreadsheet environment. *IEEE Trans. Softw. Eng.* 32 (4), 213–239.
- Schmitz, T., Jannach, D., 2016. Finding errors in the Enron spreadsheet corpus. In: IEEE Symposium on Visual Languages and Human-Centric Computing. pp. 157–161.
- Shepperd, M., Bowes, D., Hall, T., 2014. Researcher bias: The use of machine learning in software defect prediction. *IEEE Trans. Softw. Eng.* 40 (6), 603–616.
- Thota, M.K., Shajin, F.H., Rajesh, P., 2020. Survey on software defect prediction techniques. *Int. J. Appl. Sci. Eng.* 17, 331–344.
- Wahono, R.S., 2015. A systematic literature review of software defect prediction: Research trends, datasets, methods and frameworks. *J. Softw. Eng.* 1 (1), 1–16.
- Wan, Z., Xia, X., Hassan, A.E., Lo, D., Yin, J., Yang, X., 2020. Perceptions, expectations, and challenges in defect prediction. *IEEE Trans. Softw. Eng.* 46 (11), 1241–1266.
- Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F., 2016. A survey on software fault localization. *IEEE Trans. Softw. Eng.* 42 (8), 707–740.
- Wu, K., Zhao, Y., Zhu, Q., Tan, X., Zheng, H., 2011. A meta-analysis of the impact of trust on technology acceptance model: Investigation of moderating influence of subject and context type. *Int. J. Inf. Manage.* 31 (6), 572–581.
- Xia, X., Bao, L., Lo, D., Li, S., 2016. “Automated Debugging Considered Harmful” considered harmful: A user study revisiting the usefulness of spectra-based fault localization techniques with professionals using real bugs from large systems. In: 2016 IEEE International Conference on Software Maintenance and Evolution. ICSME, pp. 267–278.
- Xie, X., Liu, Z., Song, S., Chen, Z., Xuan, J., Xu, B., 2016. Revisit of automatic debugging via human focus-tracking analysis. In: Proceedings of the 38th International Conference on Software Engineering. ICSE '16, Association for Computing Machinery, pp. 808–819.

Adil Mukhtar is a scientific project assistant and Ph.D. candidate at Graz University of Technology, Austria. He received his Master's Degree in Computer Science from Lahore University of Management Technology. His research primarily focuses on fault localization, spreadsheet debugging, diagnostic systems and natural language processing based on AI enabled techniques.

Birgit Hofer works as researcher at Graz University of Technology. She received a Ph.D. degree in Computer Science (2013) and a Master's degree (2009) from the same University. Her main research interests are automatic fault localization and correction in imperative and object oriented software and spreadsheets, with a particular focus on spectrum-based fault localization, model-based debugging, and genetic programming approaches.

Dietmar Jannach is a full professor of Information Systems at the University of Klagenfurt, Austria. His main research area lies in the application of artificial intelligence technology to practical problems, with a particular focus on recommender systems in e-commerce, AI-based testing and debugging of spreadsheets, and on engineering problems of knowledge-intensive systems.

Franz Wotawa received an M.Sc. and a Ph.D. from Vienna University of Technology in 1994 and 1996 respectively. He is currently a professor of software engineering at Graz University of Technology. His research interests include model based and qualitative reasoning, theorem proving, mobile robots, verification and validation, and software testing and debugging.